



Vietnam National University of HCMC
International University
School of Computer Science and Engineering



Object Oriented Programming

Polymorphism

Instructor: Le Thi Ngoc Hanh, Ph.D

ltnhanh@hcmiu.edu.vn

Semester – Spring 2026

Lecture Objectives

- ✧ Concept of polymorphism
- ✧ To use overridden methods to effect polymorphism
- ✧ To distinguish between abstract and concrete classes
- ✧ To declare abstract methods to create abstract classes
- ✧ How polymorphism makes systems extensible and maintainable
- ✧ To determine an object's type at execution time
- ✧ To declare and implement interfaces

Introduction - Example

Scenario: A "Payment" action on E-Commerce platform:

- ZaloPay → authenticate → scanQR → process → confirm
 - BankTransfer → verify → transfer → confirm
- ☾ Same operation: pay(), BUT different implementations

Introduction - Example



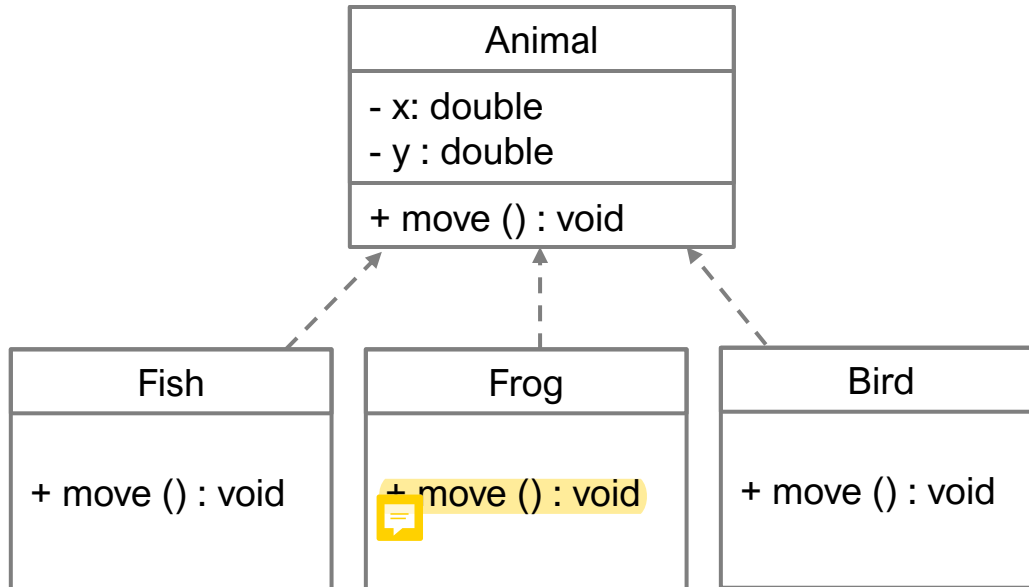
Polymorphism

- ☾ Enables you to “program in the general” rather than “program in the specific.”
- ☾ Polymorphism enables you to write programs that process objects that share the same superclass as if they’re all objects of the superclass; this can simplify programming

Polymorphism - Example

- ✧ Suppose we create a mini game that simulates the movement of several types of animals for a biological study. Classes **Fish**, **Frog** and **Bird** represent the three types of animals under investigation.
- ✧ Each class extends superclass **Animal**, which contains a method `move` and maintains an animal's current location as *x-y coordinates*. Each subclass implements method `move`.
- ✧ A program maintains an **Animal** array containing references to objects of the various **Animal** subclasses. To simulate the animals' movements, the program sends each object the same message namely, `move`.

Polymorphism - Example



Polymorphism - Example

Suppose we design a system that applies an **add()** operation to different types of values.

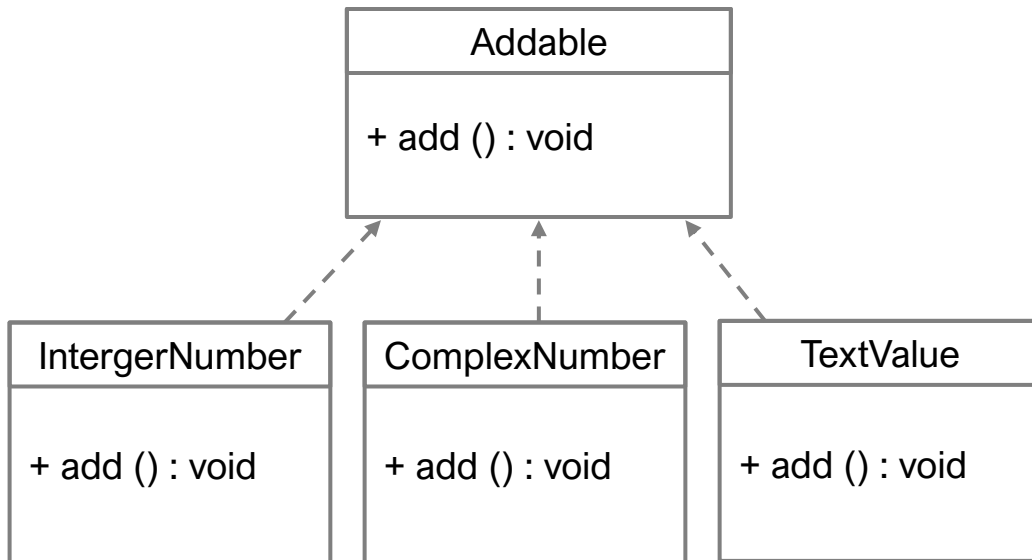
IntegerNumber performs arithmetic addition,

ComplexNumber adds real and imaginary parts, and

TextValue concatenates strings. Although the same method name **add()** is used, each object implements it differently.

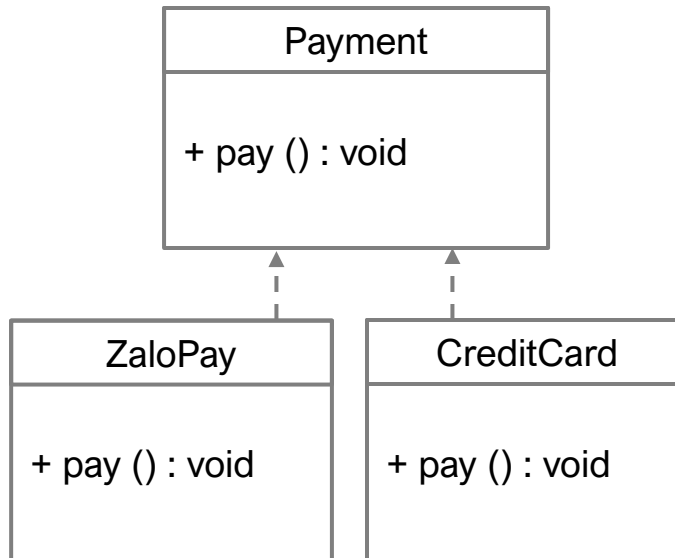
A program can process these objects through a common superclass **Addable**, demonstrating polymorphism: the same message leads to different behaviors depending on the object type.

Polymorphism - Example



Polymorphism - Example

- ★ Payment methods apply the same method `pay()`



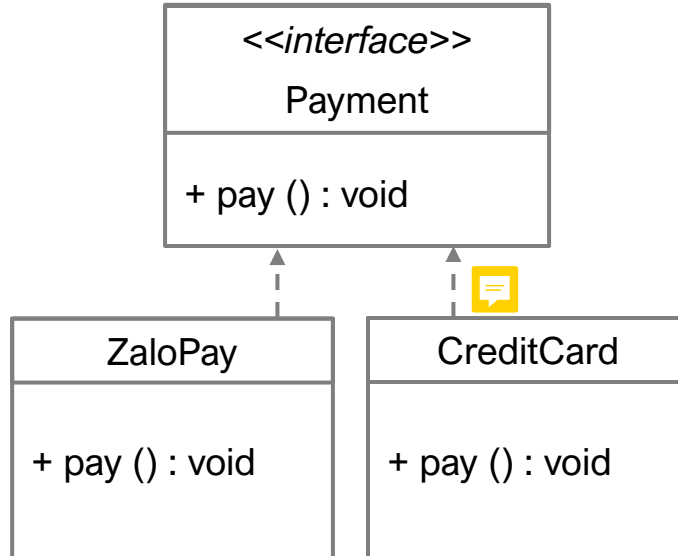
Polymorphism

- ✧ With polymorphism, we can design and implement systems that are easily *extensible*
- ✧ New classes can be added with little or no modification to the general portions of the program, as long as the new classes are part of the inheritance hierarchy that the program processes generically.
- ✧ The only parts of a program that must be altered to accommodate new classes are those that require direct knowledge of the new classes that we add to the hierarchy

Polymorphism

- Once a class implements an **interface**, all objects of that class have an *is-a* relationship with the interface type, and all objects of the class are guaranteed to provide the functionality described by the interface.
- This is true of all subclasses of that class as well.
- An interface defines a contract. Any class that implements it becomes part of the same type system. This allows us to treat different objects uniformly while still allowing each to behave differently.
- Allows objects of unrelated classes to be processed polymorphically- objects of classes that implement the same interface can respond to all of the interface method calls.

Sample code



```
interface Payment {  
    void pay();  
}  
  
class CreditCard implements Payment {  
    public void pay() {  
        System.out.println("Credit card payment");  
    }  
}  
  
class ZaloPay implements Payment {  
    public void pay() {  
        System.out.println("ZaloPay payment");  
    }  
}
```

```
Payment p1 = new CreditCard();  
Payment p2 = new ZaloPay();
```

Polymorphism

- ☾ An interface describes a set of methods that can be called on an object, but does not provide concrete implementations for all the methods.
- ☾ You can declare classes that implement (i.e., provide concrete implementations for the methods of) one or more interfaces.
- ☾ Each interface method must be declared in all the classes that explicitly implement the interface.

Polymorphism

☾ Extend to subclass

```
class PremiumZaloPay extends ZaloPay {  
    public void pay() {  
        System.out.println("Premium ZaloPay payment");  
    }  
}
```

☾ Introduce new class

```
class BankTransfer implements Payment {  
    public void pay() {  
        System.out.println("Bank transfer payment");  
    }  
}
```

Demonstrating Polymorphic Behavior

- ☾ Polymorphic behavior occurs when:
- The same reference variable
 - Invokes the same method
 - But produces different behavior depending on the actual object

```
Payment p = new CreditCard();  
p.pay();    // CreditCard behavior
```

```
p = new ZaloPay();  
p.pay();    // ZaloPay behavior
```

```
interface Payment {  
    void pay();  
}  
class CreditCard implements Payment {  
    @Override  
    public void pay() {  
        System.out.println("Pay with Credit Card");  
    }  
}  
class ZaloPay implements Payment {  
    @Override  
    public void pay() {  
        System.out.println("Pay with ZaloPay");  
    }  
}
```



```
public class Demo {  
    public static void main(String[] args) {  
        Payment p;  
        p = new CreditCard();  
        p.pay();  
        p = new ZaloPay();  
        p.pay();  
    }  
}
```

Compile-time vs Run-time

- ☾ Compile - time: Checks if method exists in reference type
- ☾ Run-time: Decides which implementation to execute

```
Payment p = new CreditCard();  
p.pay();
```

Reference type vs Object type

- ☾ Reference type = declared type of variable
- ☾ Object type = actual object in memory

```
Payment p = new CreditCard();
```

- ☾ Reference type → Payment
- ☾ Object type → CreditCard



Binding

Binding is the process of associating a method call with its implementation. When the program executes a call like `p.pay()`, the system must decide: Which class's method implementation should run?

Static Binding (Early Binding)

- Method is **resolved** at compile-time
- Based on reference type
- Used for:
 - + static methods
 - + private methods
 - + overloaded methods

Dynamic Binding (Late Binding)

- Method is resolved at runtime
- Based on actual object type
- Used for:
 - + overridden methods (polymorphism)

Abstract Classes and Methods

- ✧ An abstract class is a class declared using the keyword `abstract`.
- ✧ It is used when we want to define a general concept but not a complete implementation.
- ✧ Abstract classes are typically used as superclasses in an inheritance hierarchy.
- ✧ They cannot be instantiated because they are incomplete.
- ✧ They provide a common design and structure for subclasses.

Concrete Classes and Methods

- Classes that can be used to instantiate objects are called concrete classes.
- Such classes provide implementations of every method they declare (some of the implementations can be inherited).
- Abstract superclasses are too general to create real objects - they specify only what is common among subclasses.
- Concrete classes provide the specifics that make it reasonable to instantiate objects.
- Not all hierarchies contain abstract classes.

Abstract vs Concrete Example

```
class Employee {  
    protected String name;  
    public Employee(String name) {  
        this.name = name;  
    }  
    public double calculateSalary() {  
        return 0;  
    }  
}
```



```
Employee e = new Employee("John");  
System.out.println(e.calculateSalary());
```

Abstract vs Concrete Example

☾ Assuming that new class is derived.

```
class PartTimeEmployee extends Employee {  
    public PartTimeEmployee(String name) {  
        super(name);  
    }  
}
```

```
PartTimeEmployee p = new PartTimeEmployee("Alex");  
System.out.println(p.calculateSalary());
```

Problem!!!

Before

```
class Employee {  
    protected String name;  
    public Employee(String name) {  
        this.name = name;  
    }  
    public double calculateSalary() {  
        return 0;  
    }  
}
```

After

```
abstract class Employee {  
    protected String name;  
  
    public Employee(String name) {  
        this.name = name;  
    }  
  
    public abstract double calculateSalary();  
}
```

```
class FullTimeEmployee extends Employee {
    private double monthlySalary;

    public FullTimeEmployee(String name, double salary) {
        super(name);
        this.monthlySalary = salary;
    }

    @Override
    public double calculateSalary() {
        return monthlySalary;
    }
}

class PartTimeEmployee extends Employee {
    private int hours;
    private double rate;

    public PartTimeEmployee(String name, int hours, double rate) {
        super(name);
        this.hours = hours;
        this.rate = rate;
    }

    @Override
    public double calculateSalary() {
        return hours * rate;
    }
}
```

Abstract vs Concrete Example

★ Polymorphism at run-time:

```
public class Demo_Employee {  
    public static void main(String[] args) {  
        Employee e1 = new FullTimeEmployee("John", 1000);  
        Employee e2 = new PartTimeEmployee("Alex", 20, 10);  
  
        System.out.println(e1.calculateSalary());  
        System.out.println(e2.calculateSalary());  
    }  
}
```

Benefits of Polymorphism

- ✧ **Flexibility:** allow objects to behave as if they belong to different classes, making the code more adaptable.
- ✧ **Reusability:** facilitate code reuse by enabling a single interface or superclass method to be implemented in multiple ways by various subclasses.
- ✧ **Maintainability and Scalability:** Systems that use polymorphism require fewer code changes when new classes are added or existing ones are modified.
- ✧ **Understandability:**
By enabling modular development, polymorphism makes the code easier to understand.